

Sugar, Sugar — Documentation technique

1. Choix du langage et de la bibliothèque graphique

Python a été choisi pour sa simplicité de prototypage, son support orienté objet suffisant pour ce projet, et sa portabilité (le même code tourne sans modification sous Windows, Linux ou macOS). Aucune fonctionnalité du jeu n'exigeait la performance brute d'un langage compilé — la charge de calcul (quelques dizaines à quelques centaines de grains testés à 30 FPS) reste largement dans les capacités d'un langage interprété.

Threading et multiprocessing envisagés puis écartés

Paralléliser le test des grains (un thread ou un processus par lot de grains, par exemple) a été envisagé pour anticiper une éventuelle montée en charge, puis écarté pour deux raisons distinctes selon l'approche :

- **Threading** : en Python, le *Global Interpreter Lock* (GIL) garantit qu'un seul thread exécute du bytecode Python à un instant donné, même sur une machine multi-cœurs. Pour un travail majoritairement CPU (tester des positions, comparer des couleurs, déplacer des références dans la grille), répartir la charge sur plusieurs threads n'apporte donc **aucun parallélisme réel** — le GIL sérialise l'exécution de toute façon. Pire, le coût de synchronisation entre threads (verrous, changements de contexte) aurait probablement ralenti la boucle de physique plutôt que de l'accélérer. Le threading en Python n'a d'intérêt que pour du travail bloquant sur des entrées/sorties (réseau, disque), ce qui ne correspond à aucun besoin du jeu.
- **Multiprocessing** : cette approche contourne bien le GIL (chaque processus a son propre interpréteur), mais au prix d'espaces mémoire totalement séparés. La grille du niveau (jusqu'à 1600×900 cases, avec des références d'objets vivants dedans) devrait être sérialisée et transférée entre processus à *chaque frame* pour rester cohérente — un coût de communication qui dépasserait largement le temps de calcul qu'on chercherait à économiser, surtout à l'échelle de ce projet (quelques centaines de grains au plus). Le multiprocessing se justifie pour de gros lots de travail indépendants traités en une fois, pas pour une simulation à état partagé mise à jour en continu comme celle-ci.

Une boucle de simulation mono-thread, mono-processus, reste donc à la fois la solution la plus simple et la plus performante compte tenu de ces contraintes.

Pygame a été retenu comme bibliothèque graphique pour plusieurs raisons :

- API bas niveau simple (surfaces, blit, détection de collision par rectangles) suffisante pour un jeu 2D à base de grille, sans le surcoût d'un moteur complet (Unity, Godot) inadapté à l'échelle du projet.

- Gestion native du son (pygame.mixer), des événements clavier/souris, et du rendu, sans dépendance supplémentaire.
- Documentation et communauté importantes, ce qui réduit le risque de blocage sur un problème obscur.
- Léger à distribuer : combiné à PyInstaller, il produit un exécutable autonome sans exiger l'installation de Python chez l'utilisateur final.

2. Architecture générale

main.py → instancie Audio, Display, GameManager

src/

game_manager.py → machine à états de navigation (accueil / réglages / sélection / jeu)

game_loop.py → boucle de simulation d'un niveau (physique, dessin, entrées)

screen_manager.py → un seul poll d'événements pygame par frame, partagé entre GameManager et GameLoop

display.py → rendu, chargement des assets, mise à l'échelle/résolution

audio.py → lecture de la musique

level.py → chargement des niveaux (JSON), classe Level (état d'un niveau en cours)

sugar.py → classe Sugar (un grain)

cup.py → classe Cup (tasse, avec exigences de remplissage par couleur)

wall.py → classe Wall (obstacles JSON et murs dessinés à la souris)

portal.py → classe Portal (changeur de couleur)

save.py → persistance de la progression joueur (cache/save.json)

settings.py → constantes globales (résolution, couleurs, FPS...)

cache/

level_data.json → données de tous les niveaux

save.json → progression du joueur (créé automatiquement si absent)

assets/ → images et musiques

Le flux de navigation est une machine à états simple : GameManager boucle sur une page courante ("homepage", "setting", "level", "game") et délègue entièrement la page "game" à une instance de GameLoop, qui a sa propre boucle de simulation le temps de la partie.

3. Structures de données et objets

La grille

Chaque niveau possède une grille 2D (list de list), de la même taille que la surface logique de rendu (1600×900) : **une case de grille = un pixel logique**. Chaque case contient soit None (case vide), soit une référence directe vers l'objet qui l'occupe (Sugar, Wall, Cup, Portal). Ce choix — références d'objets plutôt que codes numériques — évite une table de correspondance séparée et garde chaque case immédiatement exploitable (isinstance(cell, Cup), etc.).

Les grains (Sugar)

Tous les grains, qu'ils soient encore en mouvement ou immobiles, sont conservés dans **une seule liste** (Level.__grains) et sont **systématiquement présents dans la grille** à leur position courante — y compris pendant qu'ils sont en train de tomber. Ce n'était pas le choix initial : une première version distinguait des grains "actifs" (jamais inscrits dans la grille) et des grains "posés" (inscrits une fois pour toutes). Ce modèle provoquait un bug après inversion de la gravité — un grain remis en mouvement pouvait rester bloqué de façon incorrecte, invisible aux grains testés la même frame que lui, faute d'inscription dans la grille pendant son déplacement. Le modèle unifié résout ce problème : chaque déplacement met à jour la grille immédiatement (ancienne case libérée, nouvelle case occupée), donc deux grains traités dans la même frame se voient correctement l'un l'autre, quel que soit l'ordre de traitement.

Les murs (Wall)

Un Wall est défini par une forme (rect, circle, triangle, ou line pour les traits dessinés à la souris) et ses paramètres géométriques. La rasterisation (conversion de la forme vectorielle en cases de grille) se fait une seule fois, au chargement ou au moment du tracé — jamais recalculée à chaque frame. Une liste séparée (Level.__walls) garde uniquement les formes d'origine, pour un rendu efficace (quelques appels pygame.draw par frame plutôt qu'un parcours de toute la grille).

Les tasses (Cup)

Une tasse occupe un carré de cases : seule la ligne du dessus (mouth_cells()) contient la référence Cup elle-même (zone d'absorption) ; les trois autres côtés sont construits comme des Wall classiques, solides. Une tasse porte un dictionnaire requirements (couleur → quantité exigée) et filled (couleur → quantité déjà reçue), ce qui permet nativement les tasses mono-couleur et bi-couleur avec la même structure.

Les portails (Portal)

Comme les tasses, un portail occupe un carré ; ses côtés gauche et droit sont solides (Wall), le dessus et le dessous restent traversables dans les deux sens. Un dictionnaire

Level.__portal_cells (position → Portal) permet de **restaurer** la référence du portail dans la grille dès qu'un grain qui le traversait s'en va — sans cela, le passage du premier grain "effacerait" progressivement le portail case par case.

Format des niveaux (JSON)

Tous les niveaux sont décrits dans un seul fichier, cache/level_data.json, sous la forme {"level-01": {...}, "level-02": {...}, ...}. Chaque niveau précise son point d'apparition, sa couleur de sucre par défaut, son quota de grains, ses obstacles, ses tasses (format simple {"color": ..., "capacity": ...} ou multi-couleur {"colors": {...}}), et ses portails éventuels.

4. Gestion de la gravité et des collisions

À chaque frame, pour chaque grain :

1. **Direction verticale** : $target_y = y + \text{gravité}$ (gravité vaut 1 ou -1, modifiable en jeu). Si cette position sort de la grille, le bord opposé est testé à sa place — la téléportation n'a lieu que si cette case est réellement libre ; sinon, le grain se comporte comme bloqué par un mur (implémente nativement les niveaux à trou dans le sol/plafond, sans code dédié à cette mécanique).
2. **Trois directions horizontales testées** à ce $target_y$: tout droit, diagonale gauche, diagonale droite (dans cet ordre, sans biais). Pour chacune :
 - hors limites latérales → grain détruit (perte sur les bords, telle que décrite dans le sujet) ;
 - case vide → candidate valide ;
 - Cup → tentative d'absorption (catch_grain) ; accepté, le grain est détruit et le remplissage incrémenté ; refusé, la direction est traitée comme bloquée ;
 - Portal → couleur du grain changée, case ajoutée aux candidates (traversable) ;
 - tout le reste (Wall, un autre grain déjà là) → bloqué.
3. Une case est choisie aléatoirement (random.choice) parmi les candidates valides ; le grain s'y déplace (Level.move_grain, qui libère l'ancienne case et occupe la nouvelle **avant** de passer au grain suivant, garantissant la cohérence intra-frame décrite plus haut). Si aucune case n'est valide, le grain reste immobile — il est retesté, sans exception, à la frame suivante.

Ce mécanisme est volontairement non optimisé (aucune distinction entre grains "en mouvement" et "au repos" ne survit d'une frame à l'autre) : à l'échelle du projet

(quelques centaines de grains maximum), le coût est négligeable, et cette simplicité évite les incohérences qu'un système de réveil sélectif introduirait.

La collision elle-même repose entièrement sur `isinstance()` explicite plutôt que sur une méthode polymorphe commune (`on_contact()` a été essayée puis abandonnée) — choix délibéré pour garder chaque cas de figure lisible d'un coup d'œil dans `GameLoop`, au prix d'un `isinstance` par type d'objet plutôt que d'un déplacement de responsabilité vers chaque classe.

5. Affichage et mise à l'échelle

Le rendu se fait sur une surface logique fixe (1600×900), indépendante de la taille réelle de la fenêtre. `Display.render()` calcule un facteur d'échelle unique (`pygame.transform.scale`, jamais `smoothscale`, pour conserver des grains d'un pixel bien nets plutôt que flous) et centre le résultat avec des bandes noires si le ratio ne correspond pas exactement. Toute position souris est reconvertie en coordonnées logiques (`screen_to_logical`) avant d'être comparée aux rectangles de collision des boutons ou utilisée pour dessiner un mur — sans cette conversion, les clics ne tomberaient plus au bon endroit dès que la fenêtre change de taille ou de résolution.

6. Sauvegarde de la progression

`cache/save.json` contient `{"player": {"level": N}}`, où `N` est le nombre de niveaux débloqués. À la victoire d'un niveau (`Level.is_complete()`), toutes les tasses pleines), `set_save(level_id)` compare la position du niveau terminé (dans l'ordre alphabétique des identifiants) à `N` : si le niveau terminé est exactement le dernier débloqué, `N` est incrémenté et le fichier réécrit. Rejouer un niveau déjà dépassé ne modifie rien, évitant tout risque de double incrémentation. Si `save.json` est absent (premier lancement, ou fichier volontairement exclu du dépôt Git), il est recréé automatiquement avec une valeur de départ.

7. Distribution

Le jeu est packagé avec `PyInstaller` (mode `--onedir`) :

```
python -m PyInstaller --onedir --windowed --name SupSugar --add-data "assets;assets"
--add-data "cache/level_data.json;cache" main.py
```

Le mode `--onedir` (plutôt que `--onefile`) a été retenu car il place l'exécutable et ses données dans un dossier permanent, jamais supprimé entre deux lancements — contrainte nécessaire pour que `cache/save.json`, généré à côté de l'exécutable, survive à la fermeture du jeu.

8. Limites connues et pistes d'amélioration

- Les réglages de résolution et de musique ne sont pas persistés d'un lancement à l'autre (contrairement à la progression des niveaux).

- GameManager et GameLoop partagent un ScreenManager commun pour la détection de clics, mais conservent chacun une portion de logique de navigation dupliquée (__click_coord) — un ScreenManager plus riche ou un véritable état de navigation partagé pourrait la réduire davantage.
- Le build ne cible que Windows ; une distribution Linux/macOS nécessiterait de relancer PyInstaller depuis une machine de l'OS cible (pas de cross-compilation possible).